Claude as a Backend

Data flow for a web UI whose open-ended actions are handled by a Claude Code session.

A small local server handles everything that has a fixed answer: saving, listing, serving files. Requests that need judgement, such as "simplify this flow" or "apply this brand to every screen", become jobs. A running Claude Code session picks up each job and does it, usually by handing it to a background subagent. Results are written to files, and the server pushes the changes to the browser.

1. Claude starts the system

There is no separately deployed service. The Claude Code session the person is working in starts both moving parts:

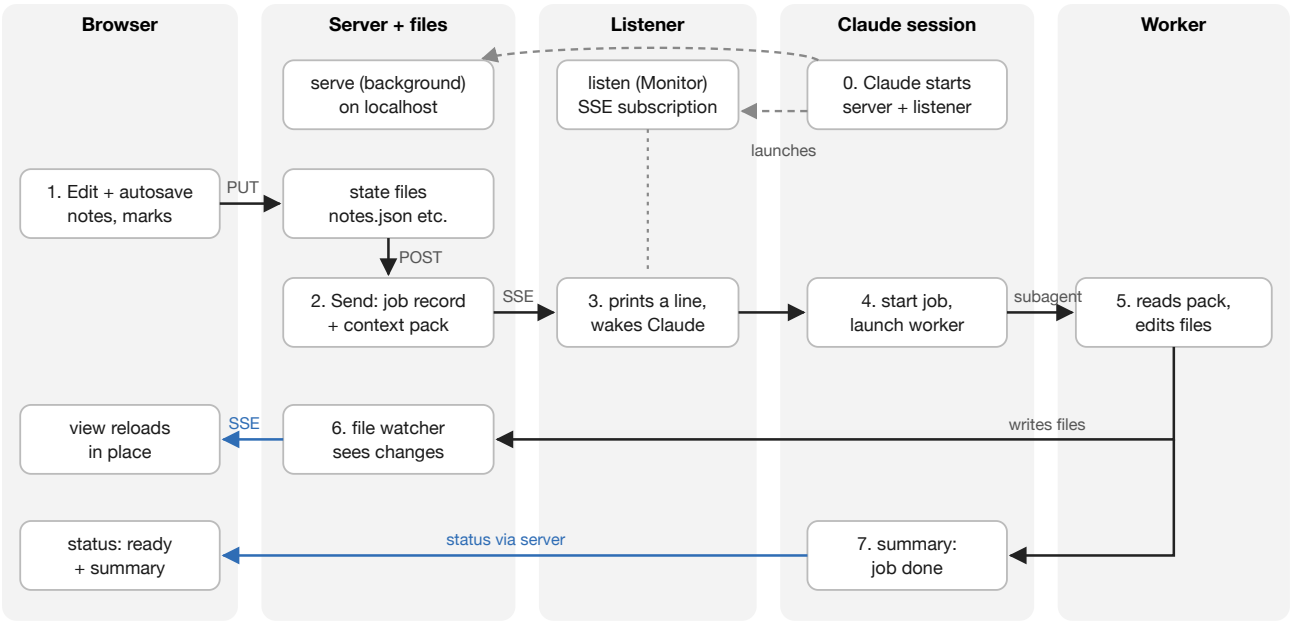
1. **The server.** Claude runs it as a background process (`serve <project>`). It binds to localhost, serves the UI, and owns the project's files: the state the UI edits, the job queue, and the artifacts being worked on.
2. **The listener.** Claude runs it under the Monitor tool (`listen <project>`). It holds an event-stream (SSE) connection to the server and prints one line per event. Each line arrives in Claude's session as a notification, so Claude is woken when there is work and idle otherwise.

Claude then gives the person the URL. The server never calls a model and holds no API key. Claude subscribes to the server, not the other way round.

2. Components

Component	Started by	Does
Browser UI	The person	Captures input (notes, marks, scoped requests), autosaves it, shows job status, reloads what changed.
Server	Claude (background process)	Persists state as files, turns sent input into jobs, serializes shared writes, watches files, pushes events to the browser and the listener.
Listener	Claude (Monitor tool)	Prints one line per server event; each line wakes the session.
Claude session	The person	Starts each job, launches a worker, reports status to the server, closes the job.
Worker	Claude (background subagent)	Does one job from its context pack and returns a one-line summary.

3. One request, end to end



Grey dashed: what Claude launches at the start. Black: the job's path. Blue: what reaches the browser.

1. **Edit.** The person makes notes or marks and scopes a request (one screen, one flow, everything). The UI autosaves to the server. No model is involved yet.
2. **Send.** The server turns the open input into jobs: one per screen, or one job that locks every screen it covers. Each job is a JSON record plus a Markdown context pack.
3. **Wake.** The server pushes the job over SSE. The listener prints it, and the line wakes the Claude session.
4. **Start.** Claude marks the job started (the server snapshots the files it may change), posts status `working`, which the server forwards to the browser, and launches a background subagent with the context pack.
5. **Work.** The worker edits the artifact files directly. Changes to shared indexes go through the server's API, which applies them one at a time, so parallel workers don't overwrite each other.
6. **Reflect.** The server's file watcher sees each change and tells the browser, which reloads only the affected view.
7. **Close.** The worker returns a one-line summary. Claude marks the job done and posts status `ready` with the summary.

4. What keeps it reliable

- **The context pack is the whole interface** between Claude and a worker: absolute paths, the files in scope, each mark with the page element under it, the rules, and the required summary line. A worker needs nothing else.

- **State lives in files.** They can be read, diffed and snapshotted by any party, and a job can be undone from its snapshot.
- **Shared writes go through the server,** and a job holds the screens it covers until it finishes.
- **Jobs persist on disk.** If no listener is connected, jobs wait and the UI keeps saving input. The next listener to connect receives them.

5. Limits

- Jobs take seconds to minutes, and each one costs model usage. The UI must treat results as asynchronous.
- A Monitor watch ends after 30 minutes, so Claude has to restart the listener. Jobs queue on disk in the meantime.
- The server and listener are background processes of the session and stop when it ends. For unattended use, the server can instead run `claude -p` for each job.
- It is built for one person on localhost, not for many concurrent users.

Reference implementation: design-driven-development, a Claude Code plugin (github.com/crathgeb/design-driven-development).